Data vault

Вадим Заигрин
Технический лидер разработки в Альфа-банке



Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и в колонках включён звук
- обновите страницу вебинара или закройте её и переподключитесь
- откройте вебинар в другом браузере
- перезагрузите ваше устройство и подключитесь снова



Поставьте в чат:

\+ если меня видно и слышно

\- если нет

Рекомендации

При просмотре с компьютера

- Используйте браузеры **Google Chrome** или **Microsoft Edge**
- Если есть проблемы с изображением или звуком, обновите страницу, нажав **F5**

При просмотре со смартфона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение

Правила участия

1. Приготовьте блокнот и ручку, чтобы записывать важные мысли и идеи
2. Продолжительность вебинара — 80 минут
3. Вы можете писать вопросы в чате
4. Запись вебинара будет доступна на платформе



Вадим Заигрин

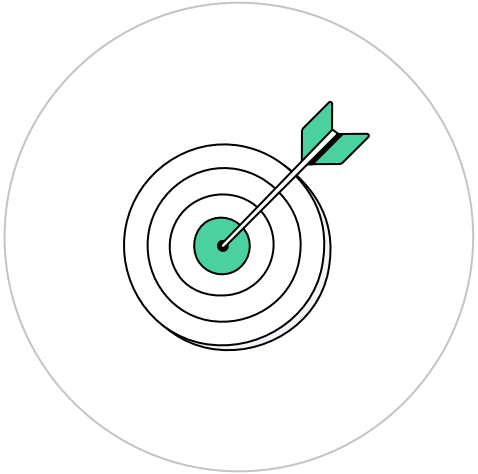
О преподавателе:

- технический лидер разработки в Альфа-Банке
- более 30-и лет в IT
- более 10-и лет в big data



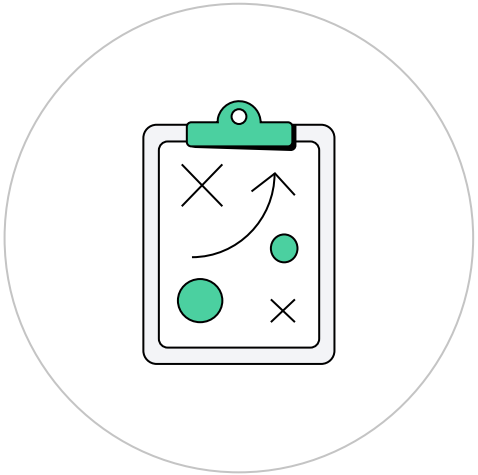
Цели занятия

- Познакомиться с методологией Data vault
- Узнать, как формируются компоненты методологии
- Рассмотреть пример построения моделирования



План занятия

1. Ограничения размерного моделирования
2. Основы Data vault
3. Компоненты Data vault
4. Пример модели



Ограничения размерного моделирования

1

Размерное моделирование, dimensional modeling

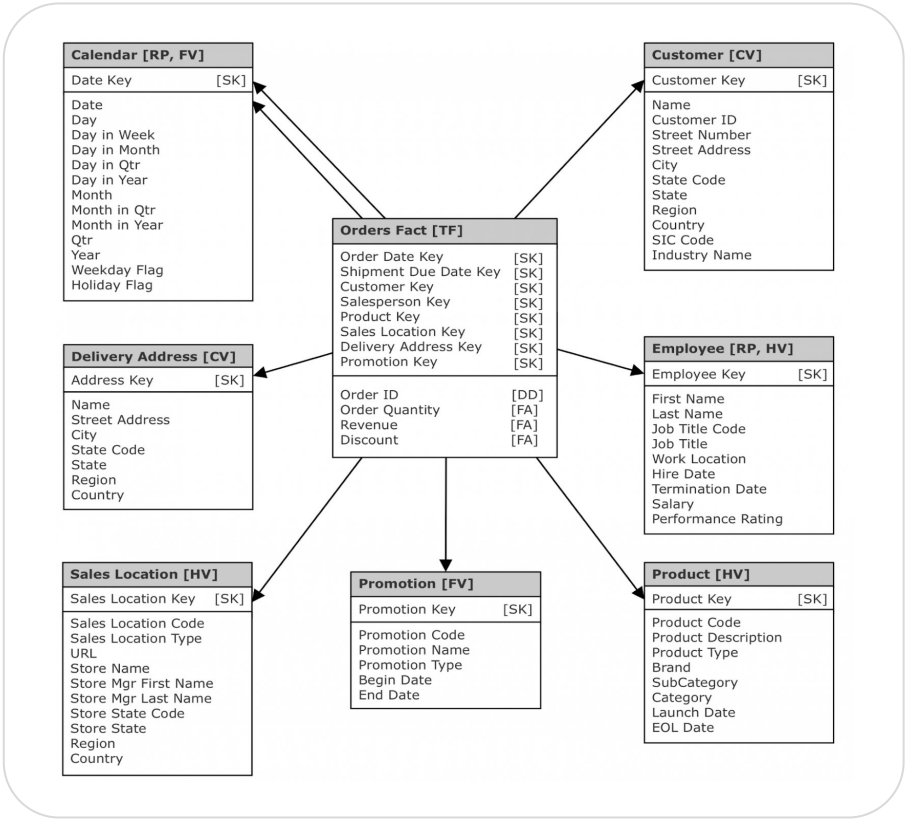
- Появилось в 1996 году как способ сделать запросы и анализ данных более эффективными: данные организуются в таблицы фактов и измерений

- **Факты** — это числовые данные, используемые для измерения чего-либо: продаж, заказов, выручки. Факты могут быть агрегированы или суммированы из соображений производительности. Например, считаются средние значения или общее количество чего-либо

- **Измерения**, measures, dimensions описывают характеристики данных, такие как время, продукт, клиент или местоположение. Обычно они представлены в виде иерархий, и каждый уровень иерархии даёт более детальное описание данных

- В этом типе моделирования данных используются две популярные схемы: «звезда» и «снежинка»

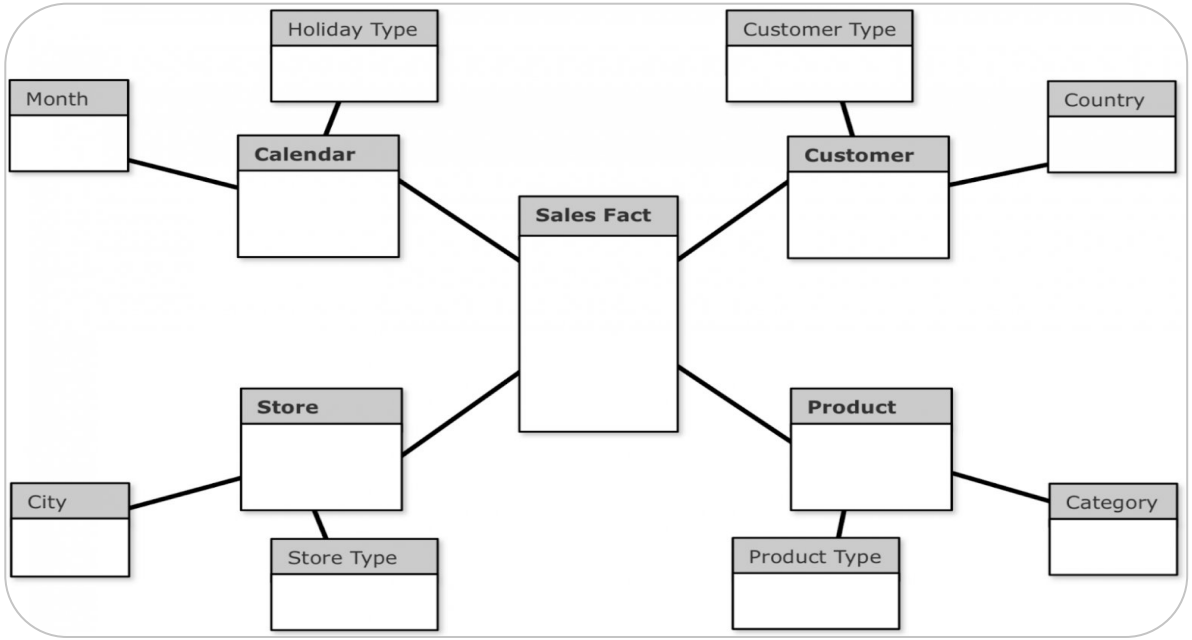
Схема «звезда»



В центре схемы «звезда» находится таблица фактов, а по бокам — несколько связанных с ней по внешнему ключу таблиц измерений

Источник: Лоуренс Корр и Джим Стагнитто, «Agile data warehouse design», 2011 г.

Схема «снежинка»



В схеме «снежинка» измерения нормализованы в отдельные таблицы

Источник: Лоуренс Корр и Джим Стагнитто, «Agile data warehouse design», 2011 г.

Проблемы и ограничения размерного моделирования

- **Сложные отношения** — предполагается, что отношения между измерениями и фактами относительно просты. Сложные взаимосвязи трудно представить в виде схемы «звезда» или «снежинка»
- **Частые изменения данных** — может плохо обрабатывать данные, которые часто меняются
- **Разреженные данные** — комбинации измерений без связанных фактов (разреженные данные) могут привести к неэффективному хранению и производительности запросов
- **Невозможно создать универсальную структуру данных**, которая обеспечит быструю обработку любого запроса
- **Проблемы при полной денормализации** — атрибут нужно обновить в нескольких местах, увеличивается объём хранилища из-за хранения избыточных данных

Мы рассмотрели

- Проблемы и ограничения размерного моделирования



Основы Data vault



Data vault — методология проектирования гибких и масштабируемых хранилищ данных.

Создана Дэниелом Линстедтом в 2000 году

Data vault

- Содержит временны́е отметки размещения данных и позволяет проследить изменение хранимой информации во времени
- **Концепция Data vault** — следствие необходимости избавиться от последовательного изменения множества связанных таблиц, когда меняются ключевые поля одной из них. Это и есть основной недостаток моделей «звезда» и «снежинка»

Data vault

В 2013 году появилась концепция **Data vault 2.0**, в которую добавлена возможность использовать хранилища NoSQL СУБД при работе с неструктурированной информацией и большими данными (big data):

- суррогатные ключи (хеш-ключи) вместо использования последовательных ключей
- бизнес-ключи — естественные ключи исходных систем
- стандарты по именованию, моделированию, загрузке и документированию хранилища данных

Архитектура Data vault 2.0

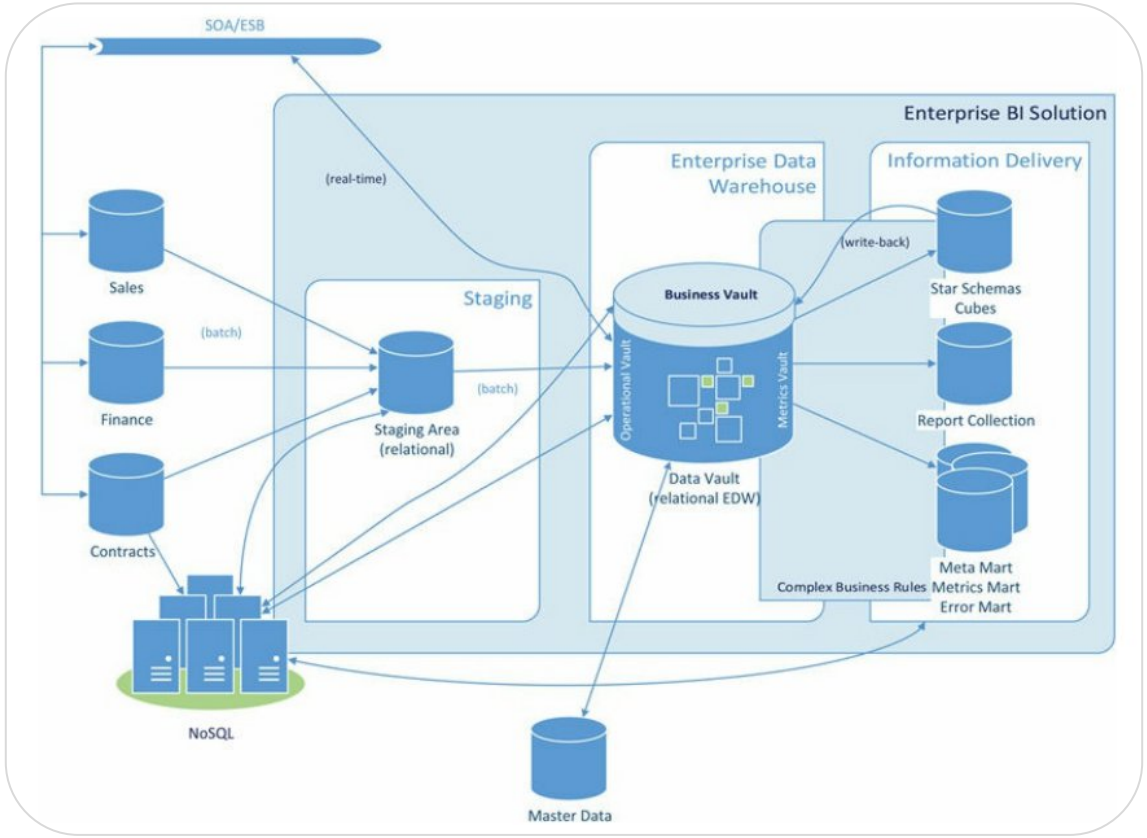
Архитектура Data vault 2.0 основана на 3 уровнях:

- **область подготовки данных**, слой Staging/ODS — собирает необработанные данные из источников
- **уровень корпоративного хранилища данных**, слой DDS — модель Data vault 2.0
- **уровень представления**, слой Data marts & information marts — аналитические витрины

Поддерживает способы загрузки данных:

- пакетную загрузку из источников
- загрузку в реальном времени через корпоративную шину данных (ESB) или сервисно ориентированную архитектуру (SOA)

Архитектура Data vault 2.0



Источник: Дэниел Линстедт и Майкл Ольшимке, «Building a scalable data warehouse with Data vault 2.0»

Отличия Data vault от других типов

3NF

- Содержат только нормализованные таблицы
- Любой запрос по выборке данных содержит срезы из множества таблиц, включая разные типы сложных соединений
- Для сложных многоуровневых хранилищ приходится использовать многоуровневые витрины

Размерное моделирование

- Относительная простота запросов и высокая скорость выборки интересующих пользователя данных
- Необходимо перестраивать множество связанных таблиц измерений при любой реорганизации информации

Отличия Data vault от других типов

Характеристики	3NF	Звезда, снежинка	Data vault
Нормализация таблиц	Обязательная	Необязательная	Необязательная
Структура хранилища	Простая	Сложная	Сложная
Объём хранимой информации	Средний или малый	Большой	Большой
Скорость доступа к информации	Средняя	Высокая	Высокая
Затраты на разработку и реорганизацию	Низкие	Высокие	Средние

Плюсы и минусы

Плюсы

- Интуитивно понятная структура, которая позволяет сформировать витрины под любые потребности бизнеса
- Простой доступ к анализу — первые верхнеуровневые отчёты доступны уже после загрузки хабов и связей
- Можно независимо разрабатывать смежные источники и легко заменять их, так как данные почти полностью отвязаны друг от друга

Минусы

- Много таблиц и джоинов, которые повышают риск ошибок и замедляют выполнение запросов
- Сложно выстраивать проверку данных из-за большого количества сущностей и сложных связей между ними
- Методология хорошо задокументирована, но требует от специалиста богатого абстрактного мышления, дополнительных навыков и знания особенностей работы специфических функций, например хеширования

Мы рассмотрели

- Концепцию метода Data vault
- Архитектуру Data vault
- Сравнение Data vault с другими методами



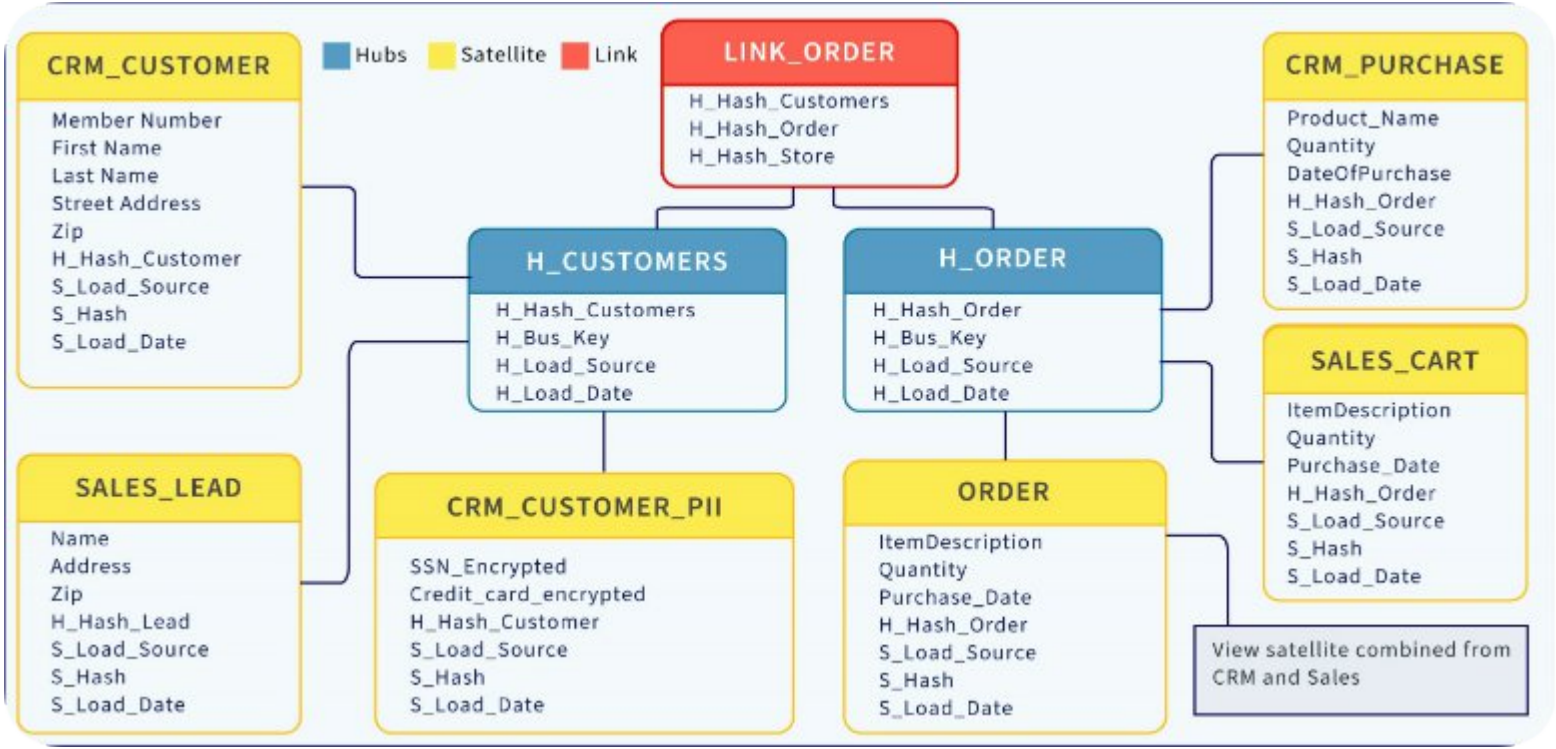
Компоненты Data vault

3

Структура Data vault

- Концепция Data vault строится вокруг 3 основных сущностей
  - **Хабы**, hubs — центральные бизнес-сущности в модели. Представляют ключевые бизнес-концепции, такие как клиенты или продукты
  - **Ссылки**, links — отвечают за моделирование транзакций, ассоциаций, иерархий и переопределение бизнес-терминов
  - **Сателлиты**, satellites — хранят описательные атрибуты о хабе или таблице-ссылке, такие как изменение данных с течением времени
- Структура хабов и ссылок на протяжении всего жизненного цикла правильно спроектированного Data vault остаётся неизменной
- При необходимости реорганизации архитектор баз данных меняет структуру сателлитов, что не затрагивает связанные таблицы

Data vault



Источник

Хабы, hubs

- **Хаб** — таблица фактов
- Основное назначение — хранить ключевую информацию о сущностях базы данных
- Состав:
  - уникальный суррогатный ключ (хеш-функция от набора натуральных ключей)
  - поля с натуральными ключами сущности (бизнес-ключи)
  - дата добавления записи
  - источник записи
- Бизнес-ключи должны быть уникальными и устойчивыми к изменениям, чтобы обеспечить точную идентификацию и отслеживание объектов
- Хеш-ключ — это первичный ключ сущности «хаб». Используется в качестве внешнего ключа для связи с таблицами-ссылками и таблицами-сателлитами

Ссылки, links

- **Таблица-ссылка** — связь между сущностями (хабами)
- Содержат ссылки на суррогатные ключи связанных хабов
- Состав:
  - хеш-ключи
  - бизнес-ключи
  - дата добавления записи
  - источник записи
  - метки активности и удаления — active_flag, deleted_flag, — опционально

Сателлиты, satellites

- **Таблицы-сателлиты** содержат все изменяемые атрибуты сущностей, включая их версии — при наличии версионности
- Сателлиты принадлежат только одному хабу или таблице-ссылке
- У хабов и таблиц-ссылок может быть сколько угодно сателлитов с разными наборами полей
- Количество и структура сателлитов определяется логической связностью и частотой изменения данных
- Один источник данных — один сателлит:
  - позволяет добавлять новые источники данных без изменения существующих таблиц-сателлитов

Сателлиты, satellites

- Рекомендуется дополнительно разделять данные по скорости изменения в них атрибутов:
  - один сателлит — для неизменяющихся атрибутов, другой — для тех атрибутов, которые меняются ежемесячно, и т. д.
- При необходимости можно создать дополнительные сателлиты под конкретные задачи без нарушения структуры Data vault

Сателлиты, satellites

- Состав:
  - родительский хеш-ключ
  - атрибуты
  - дата добавления записи
  - источник записи
  - дата начала действия, valid_from
  - дата окончания действия, valid_to
  - метки активности и удаления, active_flag, deleted_flag

Мы рассмотрели

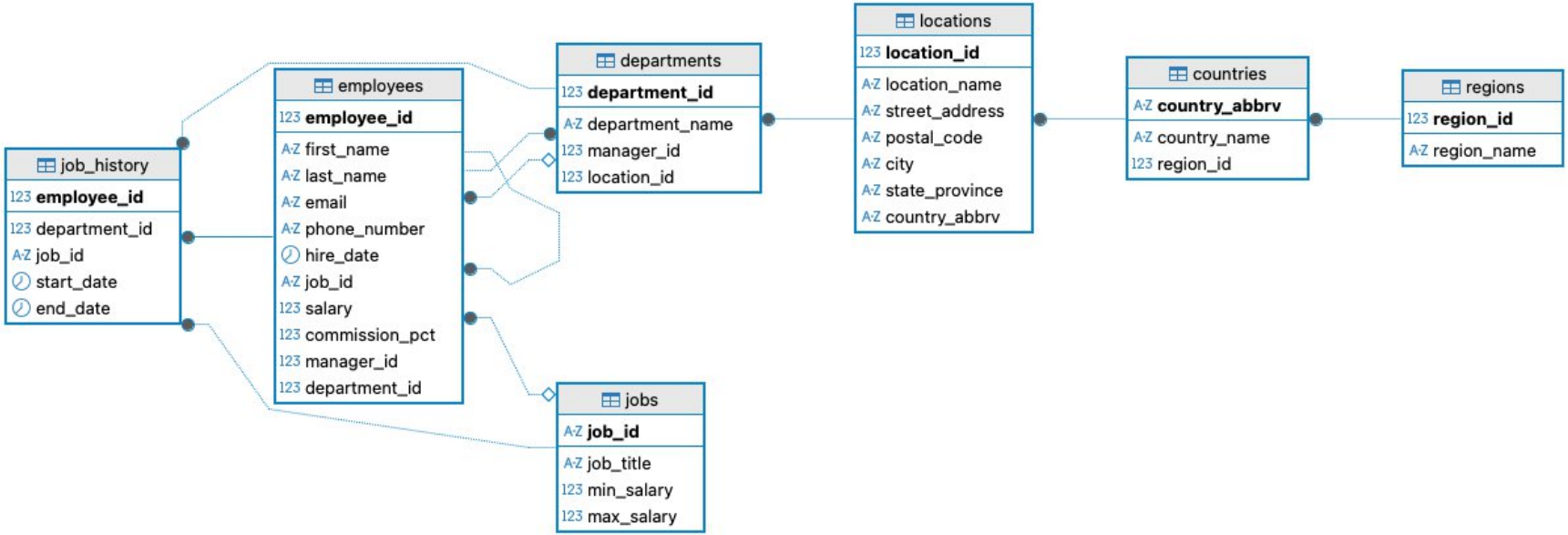
- Как формируются компоненты Data vault



Пример модели

Модель human resources, HR

- Посмотрим, как перейти от модели OLTP к модели Data vault



Источник: схема эксперта

Бизнес-ключи для хабов

- **Таблица LOCATION:**
  - PK – LOCATION_ID — просто целое число, не является бизнес-ключом
  - LOCATION_NAME — является уникальным, кандидат в бизнес-ключи
- **Таблица COUNTRIES:**
  - COUNTRY_ABBRV — бизнес-ключ
- **Таблица REGIONS:**
  - REGION_NAME — бизнес-ключ

HUB_LOCATION

- HUB_LOCATION:
  - HUB_LOCATION_KEY char(32) PK
  - LOCATION_NAME varchar(50)
  - HUB_Load_DTS date
  - HUB_Rec_SRC varchar(12)
- LOCATION_NAME — это бизнес-ключ
- LOAD_DTS — дата появления записи с этим бизнес-ключом
- REC_SRC — источник записи
- HUB_LOCATION_KEY — суррогатный ключ = md5(upper(trim(location_name)))

Что можно делать с одной HUB_LOCATION

Профилирование данных и базовый анализ с помощью бизнес-ключа, чтобы узнать:

- сколько у нас местоположений
- сколько систем-источников предоставляют нам названия местоположений
- есть ли проблемы с качеством данных
- видим ли мы названия местоположений, которые кажутся похожими, но взяты из разных источников

HUB_REGION и HUB_COUNTRY

- HUB_REGION:
  - HUB_REGION_KEY char(32) PK
  - REGION_NAME varchar(25)
  - HUB_Load_DTS date
  - HUB_Rec_SRC varchar(12)
- HUB_COUNTRY:
  - HUB_COUNTRY_KEY char(32) PK
  - COUNTRY_ABBRV char(2)
  - HUB_Load_DTS date
  - HUB_Rec_SRC varchar(12)

LNK_REGION_COUNTRY

- LNK_REGION_COUNTRY:
  - LNK_REGION_COUNTRY_KEY char(32) PK
  - HUB_REGION_KEY char(32) FK
  - HUB_COUNTRY_KEY char(32) FK
  - COUNTRY_ABBRV char(2)
  - REGION_NAME varchar(25)
  - LNK_ Load_DTS date
  - LNK_Rec_SRC varchar(12)
- LNK_REGION_COUNTRY_KEY — хеш от бизнес-ключей хабов = **md5(upper(trim(country_abbrv)) || '|' || upper(trim(region_name)))**

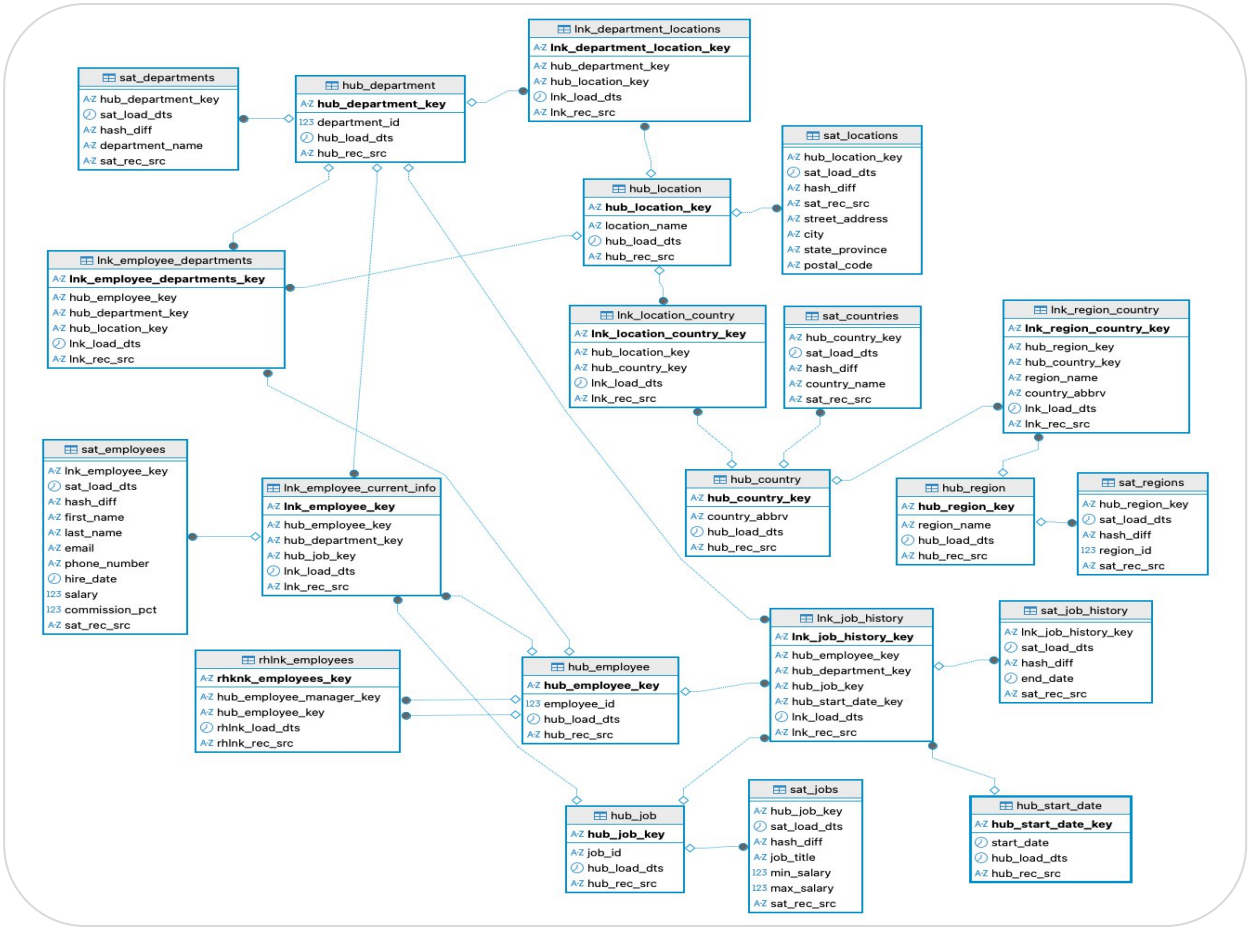
Сателлиты, satellites

- Сателлиты содержат все описательные, т. е. неключевые, столбцы и хранят историю

- SAT_LOCATION:
  - HUB_LOCATION_KEY char(32) PK FK
  - SAT_Load_DTS date
  - HASH_DIFF char(32)
  - SAT_Rec_SRC varchar(12)
  - STREET_ADDRESS varchar(40) N
  - CITY varchar(30) N
  - STATE_PROVINCE varchar(25) N
  - POSTAL_CODE varchar(12) N

- HASH_DIFF — столбец на основе хеша для определения того, изменилась ли запись = **md5(trim(street_address) || '|' || trim(city) || '|' || trim(state_province) || '|' || trim(postal_code))**

Итоговая модель



Источник: схема эксперта

Мы рассмотрели

- Пример модели, спроектированной методом Data vault



Итоги

Итоги занятия

- Обсудили ограничения размерного моделирования
- Познакомились с основами Data vault
- Изучили компоненты модели
- Рассмотрели пример моделирования





Ваши вопросы

Data vault

Вадим Заигрин
Технический лидер разработки в Альфа-банке

